

INTEGRATIVE PROGRAMMING TECHNOLOGY

ACTIVITY – MIDTERM

```
using System;
using System.Collections.Generic;

namespace EmployeeTimeTrackingSystem
{
    abstract class WorkHoursCalculator
    {
        public abstract double ComputeHours(DateTime timeIn, DateTime timeOut);
    }

    class HoursCalculator : WorkHoursCalculator
    {
        public override double ComputeHours(DateTime timeIn, DateTime timeOut)
        {
            return (timeOut - timeIn).TotalHours;
        }
    }

    class Employee
    {
        public string EmployeeNumber { get; set; }
        public string EmployeeName { get; set; }
        public string Location { get; set; }
        public DateTime TimeIn { get; set; }
        public DateTime TimeOut { get; set; }
    }

    class Program
    {
        static Dictionary<string, string> EmployeeRecord = new Dictionary<string, string>();

        static void Main(string[] args)
        {
            Console.WriteLine("Enter Employee Number: ");
            string empNum = Console.ReadLine();

            Console.WriteLine("Enter Employee Name: ");
            string empName = Console.ReadLine();

            Console.WriteLine("Enter Location (Philippines / United States / India): ");
            string location = Console.ReadLine();

            TimeZoneInfo tz = GetTimeZone(location);
            if (tz == null)
            {
                Console.WriteLine("Invalid location.");
                return;
            }

            DateTime timeIn = TimeZoneInfo.ConvertTime(DateTime.Now, tz);

            Console.WriteLine("\nTime-In Recorded:");
            Console.WriteLine(timeIn.ToString("MM/dd/yyyy hh:mm:ss tt"));
```

```

        Console.WriteLine("\nPress ENTER to Time-Out...");
        Console.ReadLine();

        DateTime timeOut = TimeZoneInfo.ConvertTime(DateTime.Now, tz);

        HoursCalculator calc = new HoursCalculator();
        double totalHours = calc.ComputeHours(timeIn, timeOut);

        string note = GenerateNote(totalHours);

        string record = $"@
Employee Number: {empNum}
Name: {empName}
Location: {location}
Time-In: {timeIn:MM/dd/yyyy hh:mm:ss tt}
Time-Out: {timeOut:MM/dd/yyyy hh:mm:ss tt}
Total Hours: {totalHours:F2}
Note: {note}
";

        EmployeeRecord[empNum] = record;

        Console.WriteLine("\n--- Employee Log ---");
        Console.WriteLine(record);
    }

    static TimeZoneInfo GetTimeZone(string location)
    {
        switch (location.ToLower())
        {
            case "philippines":
                return TimeZoneInfo.FindSystemTimeZoneById("Singapore Standard Time");
            case "united states":
                return TimeZoneInfo.FindSystemTimeZoneById("Eastern Standard Time");
            case "india":
                return TimeZoneInfo.FindSystemTimeZoneById("India Standard Time");
            default:
                return null;
        }
    }

    static string GenerateNote(double hours)
    {
        if (Math.Round(hours, 2) == 9)
            return "";

        if (hours < 9)
        {
            double remaining = 9 - hours;
            return $"Early Out. Hours left: {remaining:F2}";
        }
        else
        {
            double extra = hours - 9;
            return $"Overtime. Hours extended: {extra:F2}";
        }
    }
}

```

```
1 using System;
2 using System.Collections.Generic;
3
4 namespace EmployeeTimeTrackingSystem
5 {
6     1 reference
7     abstract class WorkHoursCalculator
8     {
9         2 references
10         public abstract double ComputeHours(DateTime timeIn, DateTime timeOut);
11     }
12
13     2 references
14     class HoursCalculator : WorkHoursCalculator
15     {
16         2 references
17         public override double ComputeHours(DateTime timeIn, DateTime timeOut)
18         {
19             return (timeOut - timeIn).TotalHours;
20         }
21     }
22
23     0 references
24     class Employee
25     {
26         0 references
27         public string EmployeeNumber { get; set; } Non-nullable property 'EmployeeNumber' must contain a non-null value when exiting constructor.
28         0 references
29         public string EmployeeName { get; set; } Non-nullable property 'EmployeeName' must contain a non-null value when exiting constructor.
30         0 references
31         public string Location { get; set; } Non-nullable property 'Location' must contain a non-null value when exiting constructor.
32         0 references
33         public DateTime TimeIn { get; set; }
34         0 references
35         public DateTime TimeOut { get; set; }
36     }
37 }
```

```
28 class Program
29 {
30     1 reference
31     static Dictionary<string, string> EmployeeRecord = new Dictionary<string, string>();
32
33     0 references
34     static void Main(string[] args)
35     {
36         Console.WriteLine("Enter Employee Number: ");
37         string empNum = Console.ReadLine(); Converting null literal or possible null value to non-nullable type.
38
39         Console.WriteLine("Enter Employee Name: ");
40         string empName = Console.ReadLine(); Converting null literal or possible null value to non-nullable type.
41
42         Console.WriteLine("Enter Location (Philippines / United States / India): ");
43         string location = Console.ReadLine(); Converting null literal or possible null value to non-nullable type.
44
45         TimeZoneInfo tz = GetTimeZone(location); Possible null reference argument for parameter 'location' in 'TimeZoneInfo Program.Get
46         if (tz == null)
47         {
48             Console.WriteLine("Invalid location.");
49             return;
50         }
51
52         DateTime timeIn = TimeZoneInfo.ConvertTime(DateTime.Now, tz);
53
54         Console.WriteLine("\nTime-In Recorded:");
55         Console.WriteLine(timeIn.ToString("MM/dd/yyyy hh:mm:ss tt"));
56
57         Console.WriteLine("\nPress ENTER to Time-Out...");
58         Console.ReadLine();
59
60         DateTime timeOut = TimeZoneInfo.ConvertTime(DateTime.Now, tz);
61
62         HoursCalculator calc = new HoursCalculator();
63         double totalHours = calc.ComputeHours(timeIn, timeOut);
64     }
```

```
63         string note = GenerateNote(totalHours);
64
65         string record = $"
66 Employee Number: {empNum}
67 Name: {empName}
68 Location: {location}
69 Time-In: {timeIn:MM/dd/yyyy hh:mm:ss tt}
70 Time-Out: {timeOut:MM/dd/yyyy hh:mm:ss tt}
71 Total Hours: {totalHours:F2}
72 Note: {note}
73 ";
74
75         EmployeeRecord[empNum] = record;    Possible null reference argument for parameter 'key' in 'string Dictionary<string, string>.this[s
76
77         Console.WriteLine("\n--- Employee Log ---");
78         Console.WriteLine(record);
79     }
80
81     1 reference
82     static TimeZoneInfo GetTimeZone(string location)
83     {
84         switch (location.ToLower())
85         {
86             case "philippines":
87                 return TimeZoneInfo.FindSystemTimeZoneById("Singapore Standard Time");
88             case "united states":
89                 return TimeZoneInfo.FindSystemTimeZoneById("Eastern Standard Time");
90             case "india":
91                 return TimeZoneInfo.FindSystemTimeZoneById("India Standard Time");
92             default:
93                 return null;    Possible null reference return.
94         }
95     }
96
97     static string GenerateNote(double hours)
98     {
99         if (Math.Round(hours, 2) == 9)
100             return "";
101
102         if (hours < 9)
103         {
104             double remaining = 9 - hours;
105             return $"Early Out. Hours left: {remaining:F2}";
106         }
107         else
108         {
109             double extra = hours - 9;
110             return $"Overtime. Hours extended: {extra:F2}";
111         }
112     }
113 }
```

OUTPUT:

```
Enter Employee Number: 101010
Enter Employee Name: Princess Jhoana B. Macatantan
Enter Location (Philippines / United States / India): Philippines

Time-In Recorded:
04/01/2026 04:35:39 PM

Press ENTER to Time-Out...

--- Employee Log ---

Employee Number: 101010
Name: Princess Jhoana B. Macatantan
Location: Philippines
Time-In: 04/01/2026 04:35:39 PM
Time-Out: 04/01/2026 04:36:17 PM
Total Hours: 0.01
Note: Early Out. Hours left: 8.99
```